

Energiesparende Funkklingel unter Berücksichtigung von Hardware, Software und Funkprotokollen

Wintersemester 25/26

**Studienarbeit, erstellt von
Meyer, Marcel
Gottlieb-Daimler-Straße 69
67663 Kaiserslautern
Matrikelnummer: 3008377
Studiengang: ETBU**

Selbständigkeitserklärung

Hiermit erkläre ich, dass ich wörtlich oder sinngemäß aus anderen Werken – dazu gehören auch Internetquellen – übernommene Inhalte als solche kenntlich gemacht und die entsprechenden Quellen angegeben habe.

Ich willige ein, dass meine Arbeit mittels einer Software auf Plagiate überprüft werden kann. Mir ist bekannt, dass es sich bei der Abgabe eines Plagiats um ein schweres akademisches Fehlverhalten handelt und dass Täuschungen nach der für mich gültigen Studien- und Prüfungsordnung geahndet werden.

Die vorliegende Arbeit habe ich selbstständig und ohne jede unerlaubte Hilfe konzipiert und angefertigt und keine anderen als die erlaubten und angegebenen Quellen und Hilfsmittel benutzt. Mir ist bewusst, dass ich Autor/in der vorliegenden Arbeit bin und volle Verantwortung für den Text trage.

Zusätzlich versichere ich, dass ich IT-gestützte oder auf künstlicher Intelligenz (KI) basierende Schreibwerkzeuge nur in Absprache mit der Betreuungsperson verwendet habe. Dabei stand meine eigene geistige Leistung im Vordergrund, und ich habe jederzeit den Prozess steuernd gearbeitet. Diese habe ich entsprechend Hilfsmittelverzeichnis mit ihrem Produktnamen und einer Übersicht des im Rahmen dieser Arbeit genutzten Funktionsumfangs vollständig aufgeführt. Sofern die zuständigen Prüfenden bis zum Zeitpunkt der Ausgabe der Aufgabenstellung konkrete KI-gestützte Schreibwerkzeuge ausdrücklich als nicht anzeigepflichtig benennen, müssen diese nicht aufgeführt werden. Ich willige ein, dass meine Arbeit mittels einer Software auf KI-Textbausteine überprüft werden kann.

Heidelberg, 27.03.2026
Ort, Datum



Unterschrift

Zusammenfassung der Tätigkeiten

Für das Retrofitting eines Bestandsgebäudes mit einer funkbasierten Klingelanlage wurden zunächst verschiedene Lösungsansätze evaluiert. Nach einer detaillierten Abwägung wurden drei kommerziell verfügbare Varianten umgesetzt. Anschließend wurden verschiedene Funkprotokolle theoretisch analysiert und eine Auswahl davon praktisch implementiert. Um den Energieverbrauch zu optimieren, wurde der Ruhestrom der Hardware-Plattformen durch das Entfernen überflüssiger Komponenten oder den Austausch gegen effizientere Bauteile deutlich reduziert.

Abkürzungsverzeichnis

ARP	Address Resolution Protocol
ELPM	Extremely Low Power Module
ESA	Energiesparende Studierendenwohnheims-Architektur
IO	Input/Output
IP	Internet Protocol
LoRa	Long Range
MAC	Media Access Control
PoE	Power over Ethernet
PPP	Point-to-Point Protocol
RPTU	Rheinland-Pfälzische Technische Universität
SoC	System-on-a-Chip
SSL	Secure Sockets Layer
TLS	Transport layer security
WAN	Wide Area Network

Inhaltsverzeichnis

1 Das Gebäude.....	4
1.1 Das Gebäude.....	5
1.2 Die bestehende Infrastruktur.....	6
2 Auswahl einer Hardware-Plattform.....	6
2.1 Empfänger.....	7
2.2 Sender.....	7
2.3 Entscheidung für einen Mikrocontroller.....	9
2.4 ESP32-basierte Development-Boards.....	10
2.4.1 ESP-Knob.....	10
2.4.2 M5Stack.....	10
2.4.3 CodeCell C3 Light.....	12
2.4.4 Boards mit Power-Latch.....	12
2.4.5 Evaluation der Ruhestrome Is der verschiedenen Geräte.....	13
3 Verbesserung der Hardware.....	14
4 Auswahl eines Funk-Protokolls.....	21
4.1 Das ISO-OSI Modell.....	21
4.2 Theoretischer Vergleich verschiedener Funkprotokolle.....	23
4.3 Evaluation des Energieverbrauchs QK der verschiedenen Funkprotokolle.....	26
5 Interpretation der ermittelten Werte.....	28
5.1 Erreichbare Lebensdauern.....	28
5.2 Ausblick: Maßnahmen zur Verlängerung der Batterielebensdauer.....	28
6 Zusammenfassung.....	29
6.1 Ergebnisse und Erkenntnisse.....	29
6.2 Herausforderungen.....	30
6.3 Ausblick und Empfehlungen.....	30
7 Fazit.....	30
8 Quellenverzeichnis.....	30

1 Das Gebäude

1.1 Das Gebäude

Das ESA (Energiesparende Studierendenwohnheims-Architektur) ist ein Wohnheim auf dem Campus der RPTU Kaiserslautern. Motiviert durch die Ölkrisen der 70er Jahre wurde es 1987 fertiggestellt. Nach einer langen Phase der intensiven Nutzung wurde es 2019 unter Denkmalschutz gestellt und umfassend saniert. Seit 2025 wird es wieder von 20 Studierenden bewohnt.



Abbildung 1: Das ESA von außen

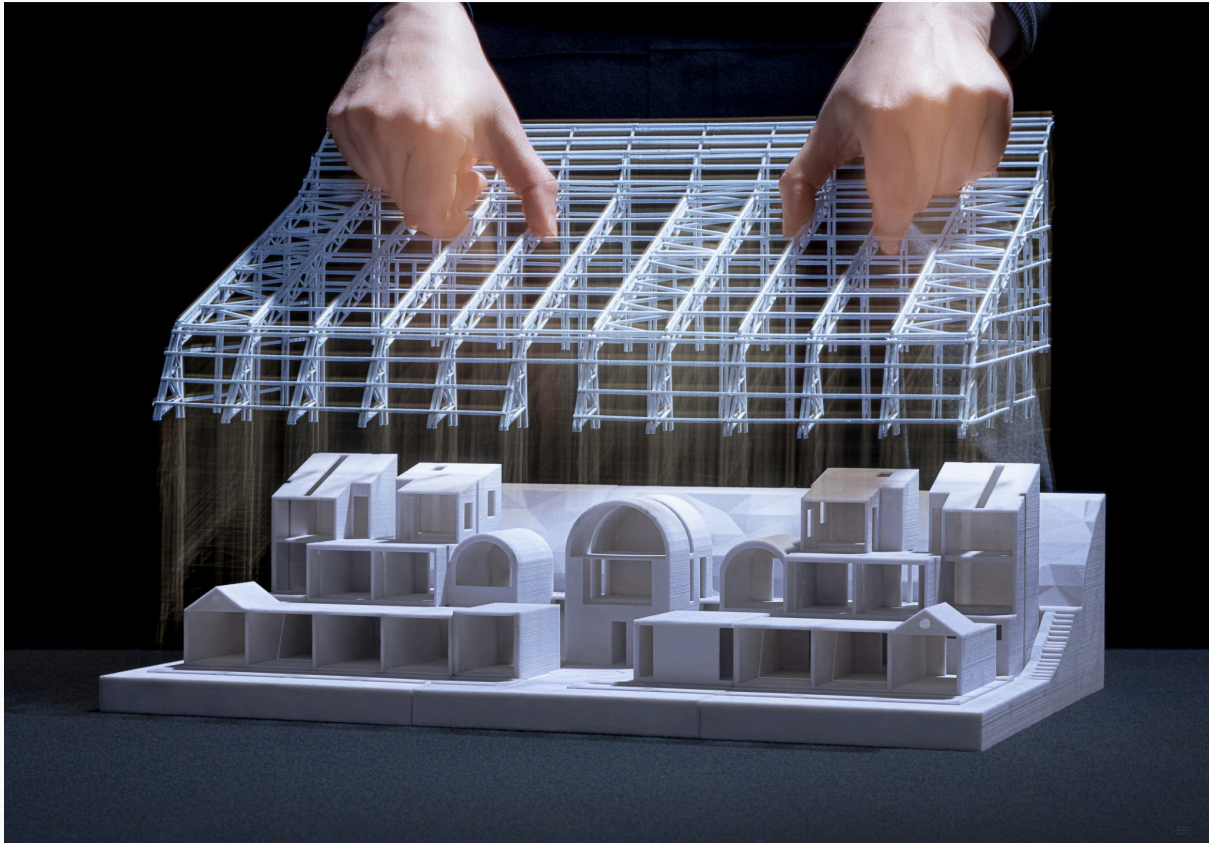


Abbildung 2: Die inneren Räume des ESA1

1.2 Die bestehende Infrastruktur

Bei der Sanierung wurde keine Klingelanlage eingebaut, weshalb Besucher bisweilen lange vor der gläsernen Eingangstür warten müssen, bis sie zufällig jemand herein lässt. Auf der Außenseite der Eingangstür gibt es weder einen Klingeldraht, noch einen 230V-Netzanschluss. Über das gesamte Wohnheim verteilt befinden sich Accesspoints, von denen man in ein offenes Freifunk-WLAN gelangen kann. In den Zimmern befinden sich Netzwerksteckdosen, sowie 230V-Netzsteckdosen.

2 Auswahl einer Hardware-Plattform

Um den Aufwand gering und das Erscheinungsbild ansprechend zu halten, wäre es gut, sowohl für die Empfängerseite, als auch für die Senderseite bereits eine möglichst integrierte Hardwareplattform zu finden. Für die Empfängerseite gilt dies umso mehr, da die Empfänger in jedem der 20 Zimmer vorhanden sein müssen.

2.1 Empfänger

Ein Empfänger für Klingelsignale muss über zwei verschiedene Fähigkeiten verfügen:

- **Funksignale empfangen:**

Die Einheiten müssen die Funksignale zuverlässig empfangen können.

- **Funksignale weiterleiten:**

Da die Distanz zwischen der Außentür und den abgelegensten Zimmern recht groß ist, werden die Empfänger in den zentraler gelegenen Zimmer außerdem die Aufgabe haben, die Funksignale an die sonst nicht erreichbaren Empfänger weiterzuleiten

- **Audiosignale wiedergeben können:**

Die Empfänger müssen, sobald sie ein Klingel-Signal empfangen haben, einen Ton abspielen können. Besser noch wäre es, wenn sie eine beliebige Audiodatei abspielen können

- **(optional) Bestätigen:**

Optional wäre es praktisch, durch einen Knopfdruck ein Bestätigungssignal senden zu können. So könnte der klingelnden Person, mitgeteilt werden, dass jemand unterwegs ist, und müsste nicht erneut klingeln. Außerdem könnten andere Empfänger darüber informiert werden, dass jemand zur Tür geht, und müssten nicht umsonst das Zimmer verlassen.

Da die Empfänger in den Zimmern selbst verbaut sind, wo Netzsteckdosen vorhanden sind, müssen sie nicht besonders energiesparend sein. Da es in den Zimmer auch Netzwerkanschlüsse gibt, wäre Power over Ethernet eine alternative Möglichkeit, die Geräte mit Energie zu versorgen.

2.2 Sender

Der Sender soll an der Außenseite der Haustür angebracht werden. Der Ort ist überdacht, aber unbeheizt. Ein Netzanschluss steht nicht zur Verfügung, genauso wenig wie ein Ethernet-Port. Ein Gerät, welches als Sender funktioniert, sollte:

- **eine ausreichend starke Sendeleistung haben:**
Dies setzt voraus, dass es über eine Sendeantenne verfügt.
- **über einen Akku verfügen:**
Da der Sender nicht an das 230V-Netz angeschlossen werden kann und auch keine anderen Kabel zur Außenseite der Haustür verlegt werden sollen, muss der Sender über einen Akku mit möglichst langer Lebensdauer verfügen. Dieser Akku sollte wiederaufladbar und/oder einfach austauschbar sein. Außerdem sollte er temperaturbeständig sein und sich möglichst wenig selbst entladen.
- **energiesparend sein:**
Der Sender sollte sich in einen Ruhemodus versetzen lassen können, um in Phasen des Nichtgebrauchs den Akku möglichst wenig zu belasten.
- **mindestens einen Knopf haben:**
Um zu klingeln, sollte ein Besucher einen Klingelknopf drücken. Dieser sollte möglichst robust sein, um nicht durch Verschleiß /unsachgemäßen Gebrauch kaputt zu gehen. Außerdem sollte er groß und augenfällig genug sein, dass er sofort als Klingelknopf erkannt wird.
- **(optional) Energy-Harvesting:**
Um das Wechseln/Aufladen des Akkus ganz zu vermeiden, wäre es geschickt, das indirekte Sonnenlicht zu nutzen, um den Akku auf einem hohen Ladestand zu halten. Ein solches Energy-Harvesting könnte nachgerüstet werden, und soll deshalb nicht Teil dieser Arbeit sein.
- **(optional) Empfänger auswählen.**
Am energiesparendsten wäre es für den Sender, wenn er durch einen Knopfdruck nur aus dem Ruhe-Modus erwachen, ein Klingelsignal abgeben und sofort wieder in den Ruhemodus zurückkehren würde. Es wäre aber auch praktisch, gezielt einen Empfänger ansprechen zu können.
 - Mit der Beschränkung auf einen Knopf könnte man das Ziel erreichen, indem man eine Art Morse-Code verwendet. Die wenigsten Menschen können heutzutage morsen, daher ist dies nicht praktikabel.
 - Man könnte einen Sender mit Touchscreen verwenden, was allerdings dem Ziel einer langen Akkulaufzeit zuwiderlaufen würde.
 - Wenn man in Kauf nimmt, dass es mehrere Knöpfe geben darf, könnte für jeden Empfänger ein einzelner Knopf verwendet werden. Bei 20 Zimmern plus Flur wären das aber mehr Taster, als die meisten Microcontroller als GPIOs zur Verfügung stellen..
 - Eine weitere Möglichkeit wäre es, die Besucher zu bitten, die Zimmernummer binär zu kodieren, wodurch man bei 20 Zimmern mit 5 digitalen Inputs auskäme.

- Diese Übersetzung könnte man den Besuchern auch abnehmen, indem man einen Priority Encoder einsetzt, um die Zimmernummer binär zu kodieren.
- Als weitere Möglichkeit käme noch in Frage, die Zimmernummer mit einem Schiebepotentiometer und einer Skala abzufragen, wobei der μC die gewünschte Zimmernummer mittels ADC dekodieren müsste.

Da alle diese Herangehensweisen den Rahmen dieser Studienarbeit sprengen würden, sollen sie nicht Teil dieser Arbeit sein. Stattdessen wird sich auf einen Knopf beschränkt, der alle Empfänger anklingelt.

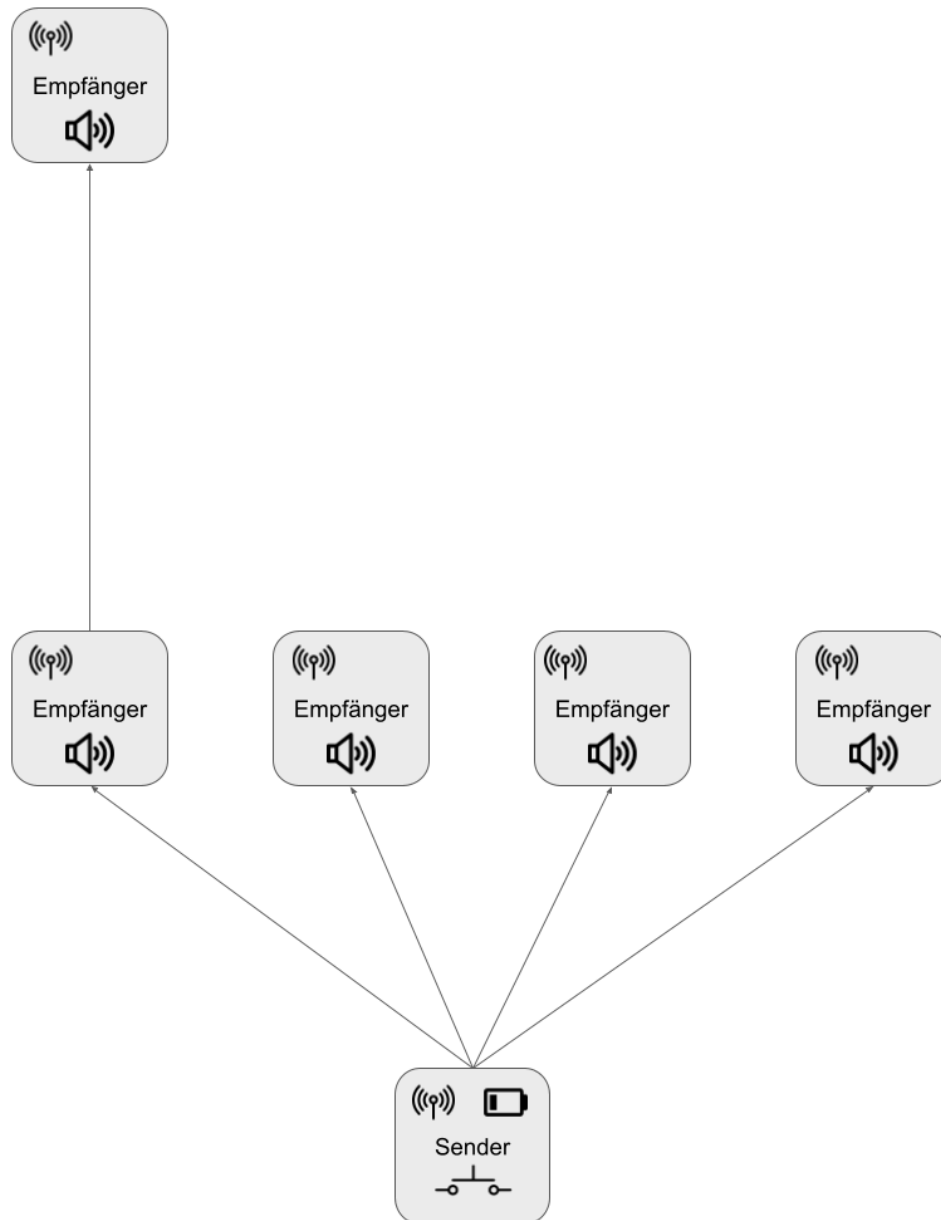


Abbildung 3: Ein Überblick über das Klingelsystem aus Sendern und Empfängern

2.3 Entscheidung für einen Mikrocontroller.

Aufgrund der Anforderungen an einen Ruhemodus, sowie an die Fähigkeit zu verschiedenen Funkstandards wurde sich für ein ESP32-basiertes System entschieden. Dieser Mikrocontroller verfügt mit seinem Deep-Sleep-Modus über einen sehr geringen Stromverbrauch von 10 μA .¹ Es gibt verschiedene Varianten des ESP32, wobei fast alle Wifi-

und Bluetooth-fähig sind. Außerdem gibt es für ESP32-Mikrocontroller das ESP-NOW-Protokoll, welches für energiesparende Funkkommunikation sehr vielversprechend ist. Einige ESP32-Boards verfügen auch noch über andere Drahtlos-Fähigkeiten, wie Zigbee oder Thread.

Auch ist der ESP32 in der Hobby-Elektroniker-Community weit verbreitet, weshalb es viel quelloffenen Code und eine Vielzahl an Hardware-Boards gibt. Nach Möglichkeit soll sowohl der Sender als auch der Empfänger ESP-32-basiert sein, um den Entwicklungsaufwand gering zu halten. Der Vollständigkeit halber seien hier noch ein paar weitere Optionen gelistet:

- **nrf52-Serie**

Die nRF52-Serie ist eine Familie von Funk-Mikrocontrollern von Nordic Semiconductor, die speziell für drahtlose Anwendungen mit niedrigem Energieverbrauch entwickelt wurden². Sie werden vermehrt in IoT-Geräten, Wearables und Smart-Home-Produkten eingesetzt, bei denen Bluetooth Low Energy (BLE), Thread, Zigbee oder andere drahtlose Protokolle zum Einsatz kommen. Sie verfügen allerdings nicht über Wifi-Fähigkeiten und die Entwicklung mit ihnen ist nicht ganz so niederschwellig, da sie in der Bastler-Community nicht sehr verbreitet sind

- **Raspberry Pi pico w**

Der Raspberry pi pico w ist ein development board mit einem RP2040 Mikrocontroller, sowie einem Infineon CYW43439 Wireless-Chip. Der Wireless-Chip ist sowohl Bluetooth- als auch Wifi-fähig. Der RP2040 verfügt über einen Deep-Sleep-Modus, dieser verbraucht mit 180µA bis 1000µA³ jedoch deutlich mehr als der ESP32¹.

- **µC + externer wireless chip**

Durch Entkopplung von Mikrocontroller und externem wireless-Chip gewinnt man an Flexibilität, wobei der Entwicklungsaufwand allerdings steigt. Man könnte einen beliebigen, möglichst stromsparenden Mikrocontroller mit einem Wireless Chip, zum Beispiel von Nordic Semiconductor oder Silicon Labs nutzen. Da hier aber verschiedenste Funktechniken und Protokolle untersucht werden sollen, müssten zumindest in der Entwicklungsphase sehr viele Prototypen entwickelt werden, was den Entwicklungsaufwand gegenüber einer integrierten Lösung erhöhen würde.

2.4 ESP32-basierte Development-Boards

2.4.1 ESP-Knob

Die Suche nach einer geeigneten Hardware-Plattform begann bei einem Blog-Post, welcher auch die eigentliche Inspiration für das Projekt war:

Darin wurde der Der ESP-Knob⁴ vorgestellt, ein minimalistisches IoT-Gerät, welches lediglich aus dem ESP32-H2, sowie einem Drehencoder mit Taster-Funktion besteht. Hinzu kommen notwendige Peripherie-Bauteile wie Spannungswandler und Lade-ICs für den Akku. Es stellte sich leider heraus, dass der ESP-Knob als Konzeptstudie gedacht war und nicht kommerziell erhältlich ist.

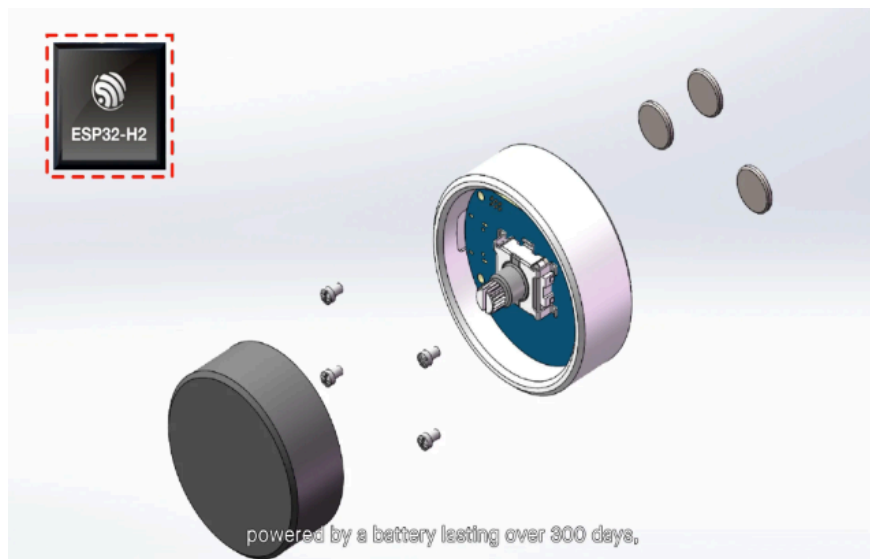


Abbildung 4: Der ESP-Knob4

2.4.2 M5Stack

Die M5Stack-Serie ist eine Reihe ESP32-basierter modularer Hardware-Bausteine, die sich wie Legosteine zu IoT-Geräten zusammenbauen lassen. Im Sortiment gibt es Displays, Akkus, Compute-Bausteine, Sensoren, Aktoren etc. Ein Baustein, der sich besonders für dieses Projekt eignet, ist der M5Stack Atom. Es ist ein kleiner Compute-Baustein, der bereits über einen Taster verfügt. Es gibt ihn in verschiedenen Versionen, die im folgenden zusammengefasst sind:

	Atom Lite	Atom S3Lite	Atom Echo
ESP-32 μ C	ESP32-C3	ESP32-S3	ESP32-C3
Taster	Ja	Ja	Ja
Taster an RTC-IO	Ja	Nein	Ja
Lautsprecher	Nein	Nein	Ja
Akku	Nein	Nein	Nein

Native-USB	Nein	Ja	Nein
USB-C	Ja	Ja	Ja

Aufgrund der Integration eines Lautsprechers mit dem ESP32 in einem kleinen Gehäuse ist der M5Stack Atom Echo sehr gut geeignet, um die Hardwareplattform des Empfängers zu stellen. Sie können mit einem USB-C Netzteil, oder mit einem M5-Stack PoE-Adapter⁵ mit Strom versorgt werden.

Beim Atom S3Lite ist der Taster nicht an einen RTC-IO angeschlossen. Das ist insofern problematisch, dass der ESP32 nur durch einen RTC-IO aus dem Deep-Sleep-Modus aufgeweckt werden kann. Um den Atom S3Lite als Sender nutzen zu können, müsste also erst der Taster mit einem RTC-IO verbunden werden, was den Entwicklungsaufwand etwas erhöht.

Allem Anschein nach würden sich aber sowohl der M5Atom S3Lite, als auch sein Vorgänger, der M5Atom Lite für den Sender eignen. Von beiden wurde ein Exemplar bestellt, um es auf seine Praxistauglichkeit näher zu untersuchen. Zusätzlich wurde je ein Exemplar der M5Stack Batteriemodule Atomic Battery Base⁶ und TailBat⁷ bestellt, um den Sender mit Strom zu versorgen.



Abbildung 7: Der M5Stack Atom Lite, M5Stack Atom S3Lite und M5Stack Atom Echo^{8–10}

2.4.3 CodeCell C3 Light

Eine weitere vielversprechende Plattform stellt das Modell CodeCell C3Lite dar. Es vereint einen ESP32-C3 mit der nötigen Elektronik, um einen LiPo-Akku über USB-C aufzuladen. Es verfügt zwar nicht über ein Gehäuse oder einen eingebauten Taster, dafür ist er mit 18.5mm × 18.5mm äußerst kompakt. Er wurde ebenfalls bestellt, um ihn mit den M5Stack-Geräten zu vergleichen.



Abbildung 8: Der CodeCell C3 Light¹¹

2.4.4 Boards mit Power-Latch

Keines der oben genannten Geräte ist explizit auf minimalen Stromverbrauch im Deep-Sleep-Modus abgestimmt. Es handelt sich um Allround-Geräte, die teilweise mit LEDs und Sensoren bestückt sind. Eine optimale Hardware-Plattform würde wirklich nur den ESP32- μ C und einen Taster beinhalten. Alle benötigten Peripherie-Bauteile wie LDOs, Spannungswandler, Laderegler sollten einen so niedrigen Ruhestrom haben, dass er den Ruhestrom des ESP32- μ C nicht signifikant erhöhen.

Nach etwas Recherche stellte sich heraus, dass es Ansätze in diese Richtung gibt:

- **Objex Link:**

Objex Labs ist ein italienisches Startup. Entwickelt werden Low-Power Development-Boards, sowie integrierte Module, die sogenannten Extremely Low Power Modules (ELPMs). Die Development-boards werden auf der Onlineplattform Tindie verkauft, wo sie derzeit (11.02.2026) ausverkauft sind. Die ELPMs werden auf der Seite hentrionics.eu verkauft. Die ELPMs verfügen nicht über eine integrierte Schaltung zum Laden von Akkus, weshalb es ein größerer Aufwand wäre, sie in einem Sender zu verbauen.



Abbildung 9: Der Objex Link12

- **Picoclick:**

Ein fertiges Low-Power Development-board mitsamt Akku-Lade-IC und USB-C Anschluss ist der Picoclick-C3, entwickelt von dem deutschen Youtuber makermokoe.

Es misst 10mm × 18mm, verfügt über einen ESP32-C3, und einen robusten Taster sowie eine Onboard-Chip-Antenne. Es wird ebenfalls auf der Plattform Tindie angeboten und ist dort ebenso ausverkauft.

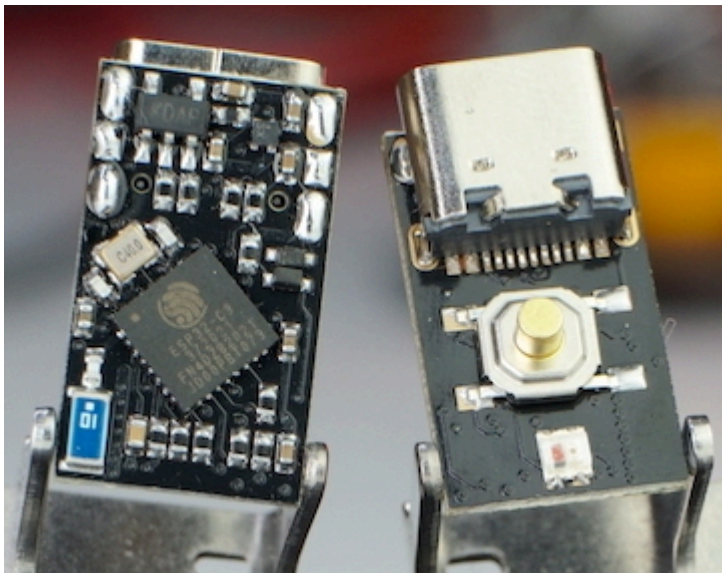


Abbildung 10: Der Picoclick13

Beiden Plattformen ist gemein, dass sie neben hocheffizienten Spannungswandlern und Lade-ICs auf einen sogenannten Power-Latching-Circuit setzen. Hierbei schaltet der ESP32 seine eigene Stromversorgung ab, indem er z.B. den Enable-Eingang des 3.3V-LDO deaktiviert. Durch Drücken eines Tasters wird dieser Enable-Eingang dann kurzzeitig wieder aktiviert. Der ESP bootet und aktiviert den Enable-Eingang selbst, sodass dieser aktiviert bleibt, auch wenn der Taster wieder losgelassen wird. Jetzt kann der ESP ein Programm abarbeiten und anschließend den Enable-Eingang des 3.3V-LDO wieder deaktivieren, wodurch er komplett abschaltet. Im Beispiel des Objex ELPM kann der Standby-Stromverbrauch des Moduls so auf 100 nA gesenkt werden.⁸

2.4.5 Evaluation der Ruhestrome I_s der verschiedenen Geräte

Zum Vergleich der Ruhestrome wurden alle Hardwareplattformen in den Deep-Sleep-Modus versetzt. Mit einem Power Profiler Kit von Nordic Semiconductors⁹ wurden die Development mit Strom versorgt und dabei die Ruhestrome der verschiedenen Development Boards gemessen.

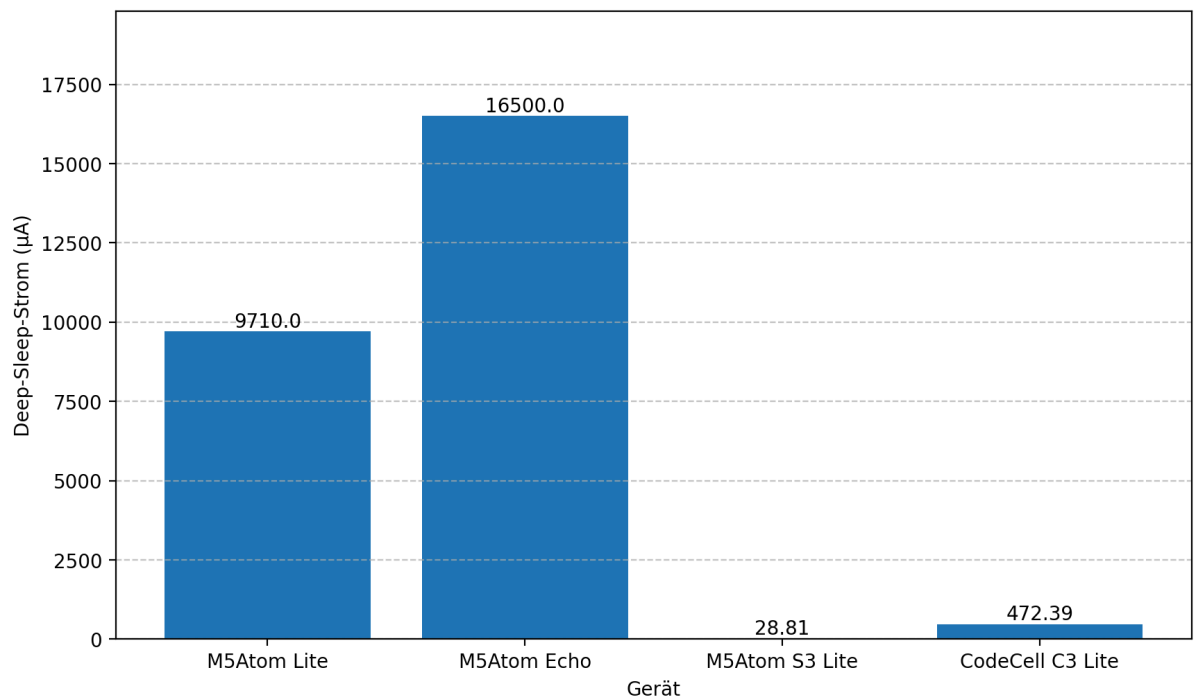


Abbildung 11: Die Ruhestrome verschiedener ESP32-basierter Hardware-Plattformen im Vergleich

Man sieht, dass die Ruhestrome der verschiedenen Geräte sehr unterschiedlich sind, obwohl sie ja alle über einen ähnlichen Mikrocontroller verfügen. Der M5Stack Atom S3 Lite liegt mit seinen 28.81 µA als einziger in der Nähe der für den ESP32 angegebenen 10 µA¹. Die anderen Geräte verbrauchen ein Vielfaches davon, weshalb sie so nicht für einen akkubetriebenen Sender in Frage kommen.

3 Verbesserung der Hardware

Weder die Geräte der M5Stack-Serie, noch der CodeCell C3Light sind auf maximales Energiesparen ausgelegt. Dies merkt man an den folgenden Features, die für einen Klingelknopf unnötig sind, den Stromverbrauch aber erhöhen:

- M5 Atom Lite

- Eine RGB LED vom Typ SK6812, mit einem Ruhestrom von 1 mA sollte entfernt werden.
- Ein CH552 Mikrocontroller als Serial-USB Converter. Dieser könnte zwar in einen Schlafmodus mit $I_s = 8\mu A$ versetzt werden, dafür müsste aber ein externer Oszillator vorhanden sein, was auf dieser Platine nicht der Fall ist. Möglicherweise liegt der Stromverbrauch eher bei den im Datenblatt für den light sleep modus angegebenen 100 μA . Dieses Bauteil könnte zwar entfernt werden, dann ließe sich der ESP32 allerdings nicht mehr über USB programmieren, weshalb dies erst ganz am Ende der Entwicklung geschehen sollte.
- Als DC-DC Wandler kommt ein SY8089 Buck Converter zum Einsatz. Dieser gibt für einen Ausgangsstrom von 10mA noch eine Effizienz von 88% an, für niedrigere Ströme gibt es keine Angaben. Es kann aber davon ausgegangen werden, dass die Effizienz bei geringeren Strömen noch geringer ist. Außerdem weist der SY8089 einen Ruhestrom von 55 μA auf. Da dieser DC-DC Wandler den Strom für das gesamte Modul bereitstellt, wird der gesamte Stromverbrauch, den die Batterie "sieht", mit einem Faktor a multipliziert. Für einen 100% effizienten Buck Converter wäre $a < 1$, wenn der Buck Converter allerdings mehr Strom verbrauchen würde, als er auf der Niederspannungsseite bereitstellt, wäre a möglicherweise > 1 , weshalb hier ein linearer Spannungsregler (LDO) vermutlich vorzuziehen gewesen wäre. Ein LDO mit passender Bauform wurde leider nicht gefunden. Man könnte zwar versuchen, einen TPS7A20¹⁰ von Texas Instruments einzubauen, müsste aber den GND-Anschluss neu verdrahten, was auf der kompakten Platine sicher nicht einfach wäre.
- Hinzu kommt (was hier aber nicht geändert werden kann), dass die M5Stack-Geräte die Batteriespannung in ihren Batteriemodulen mit einem Boost-Converter auf 5V hochwandeln, nur um sie dann für die ESP32-Bausteine wieder herunterzuwandeln. Dieser zweite DC-DC-Wandler hat bei niedrigen Strömen vermutlich eine ebenso schlechte Effizienz, was die Batterie weiter belastet.
- Am Ausgang eines VCC-Supervisory Circuits befindet sich eine generische TVS-Diode, die den Mikrocontroller resettet soll, falls eine zu hohe Eingangsspannung anliegt. Da hierfür keine nähere Typenbezeichnung vorliegt, wird der Stromverbrauch dieses Bauteils anhand einer generischen TVS-Diode¹¹ zu 10nA geschätzt. Sie zu entlöten würde sich wohl kaum lohnen.
- Der VCC Supervisor Circuit SGM803 soll den Mikrocontroller ebenfalls resettet, wenn eine zu niedrige Versorgungsspannung

detektiert wird. Er hat laut Datenblatt einen Ruhestrom von $13\mu\text{A}$. Da er für ein Funktionieren der Klingel nicht unbedingt notwendig ist, kann er auch entfernt werden.

- M5Atom S3Lite
 - Die WS2812 LED besitzt einen deutlich niedrigeren Ruhestrom, wird aber nicht benötigt, und kann also entfernt werden
 - Der GS321 Operationsverstärker sorgt mit seiner Beschaltung dafür, dass der ESP32 in den Bootmodus gelangt, sobald der Reset-Button länger gedrückt wird. Dieser OpAmp wird von der Firma Gainsil produziert, und in verschiedenen Datenblättern finden sich verschiedene Angaben über Ruhestrome von $40\mu\text{A}$ (typical) bis $400\mu\text{A}$ ¹². Im Deep Sleep wurde für den gesamten S3Lite nur ein Verbrauch von $28\mu\text{A}$ gemessen, also scheint der Ruhestrom des GS321 geringer zu sein als angegeben. Jedenfalls ist der Opamp für das Flashen des ESP32 notwendig und sollte deshalb erst zu allerletzt entfernt werden.
 - Außerdem verfügt der S3Lite über einen Leistungsschalter, mit dem ein externes LCD Backlight versorgt werden kann. Da dieser nicht benötigt wird, kann er ebenfalls entfernt werden.
 - Der M5Atom S3Lite ist für einen energiesparenden Betrieb recht gut aufgestellt. Leider ist sein Push-Button an IO 47 angeschlossen. Da das kein RTC-Pin des ESP32 ist, kann der ESP32 nicht über diesen Pin aus dem Deep-Sleep-Modus geweckt werden. Es muss also dieses Signal auf einen anderen Pin gelegt werden, sonst lässt sich der Deep Sleep Modus hier nicht nutzen.
- CodeCell C3 Light
 - Die SK6805 RGB-LED hat einen Ruhestrom von $300\mu\text{A}$ und kann entfernt werden.
 - Der VCNL4040-Ambient Light und Proximity-Sensor verbraucht angeblich nur $0.2\mu\text{A}$, wenn die Umgebung dunkel ist. Da er aber nicht gebraucht wird, kann auch er entfernt werden.
 - Vor dem LDO befindet sich eine ESD-Schutzdiode mit einem Ruhestrom von $1\mu\text{A}$
 - $15\mu\text{A}$ fließen dauerhaft über einen Spannungsteiler aus zwei $100\text{k}\Omega$ -Widerständen. Dadurch kann der ESP32 die Batteriespannung überwachen. Dieses Feature ist zwar recht nützlich, aber der Strom ist etwas zu hoch. Vielleicht könnte man hier noch hochohmigere Widerstände verwenden, um den Stromverbrauch zu reduzieren.
 - Der LDO selbst, ein NCP117 von onsemi, kann bis zu 500mA bereitstellen und verfügt über einen Ruhestrom von $60\mu\text{A}$. Er soll durch einen NCP167BMX330TBG ersetzt werden. Dieser hat den

gleichen Footprint, kann bis zu 700mA bereitstellen und hat einen Ruhestrom von 16 μ A.

In der folgenden Tabelle werden diese Bauteile zusammengefasst, wobei unbedingt zu entfernende Bauteile rot markiert sind, während Bauteile, die vielleicht entfernt/ersetzt werden sollen, gelb markiert sind.

Bauteil	M5 Atom Lite	M5 Atom S3Lite	CodeCell C3 Light
Onboard LED	SK6812 ¹³ , 1000 μ A ¹⁴	WS2812C ¹⁵ , 1 μ A ¹⁶	SK6805 ¹⁷ , 300 μ A ¹⁶
Serial-USB-Conve rter	CH552 ¹⁸ , 100 μ A ¹⁷		
Opamp		GS321 ¹⁹ 430 μ A ²⁰	
DC-DC	SY8089 ¹⁸ , 55 μ A ²¹	SY8089 ¹⁵ , 55 μ A ²¹ C	
Proximity and Ambient Light Sensor			VCNL4040 ²² , 0.2 μ A ²³
Zener Diode	TVS/BiDir, 10nA(estimated)	ESD5Z3V3 ¹⁸ , 0.05 μ A ²⁴	SP3031 ²² , 1 μ A ²⁵
Resistive Battery monitor			2*100k ²² , 15 μ A
LCD Driver		SGM2578 ¹⁵ , 1.5 μ A ²⁶	
VCC Supervisor	SGM803 ¹⁵ , 13 μ A ²⁷		
LDO			NCP177 ²² , 60 μ A ²⁸

Die beschriebenen Änderungen wurden nach und nach durchgeführt und nach jeder Änderung der neue Ruhestrom gemessen:
TODO

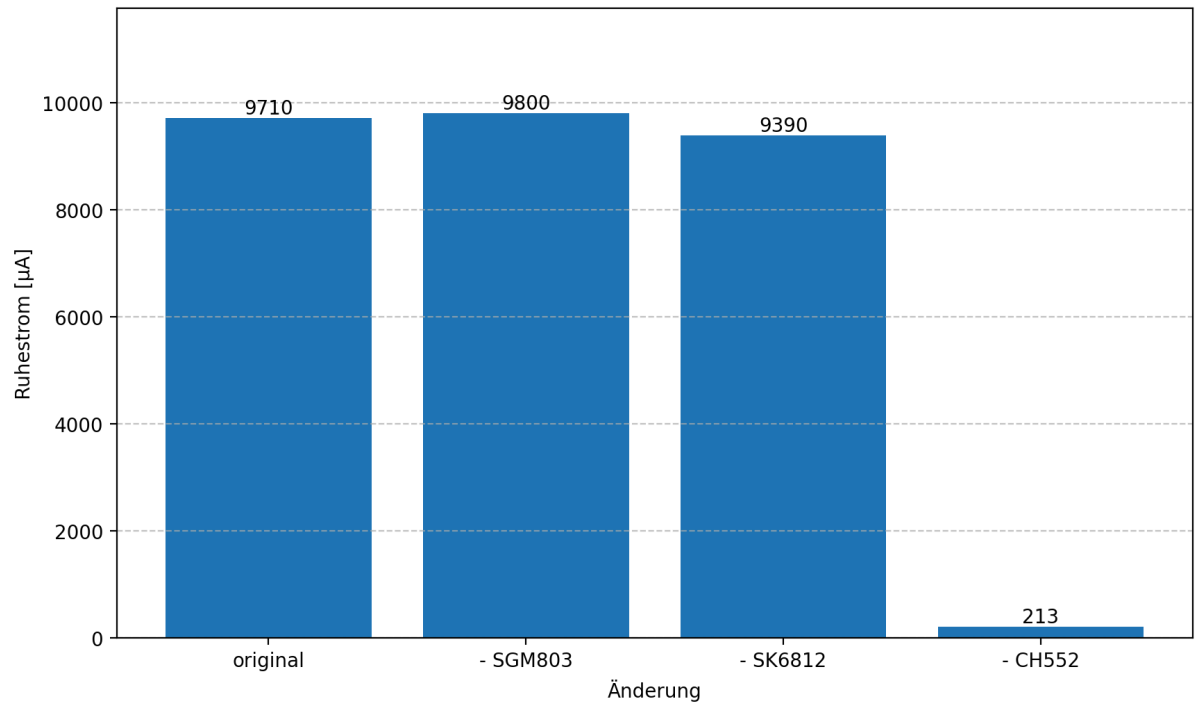


Abbildung 12: Hardwaremodifikationen des M5Stack Atom Lite

Im Falle des M5Stack Atom Lite machte die Entfernung des VCC Supervisory ICs keinen Unterschied für den Energieverbrauch. Die Entfernung der RGB-LED verringerte den Ruhestrom um ca 410 µA. Der Koprozessor, der hier als Serial-to-USB-Converter dient, hatte einen großen Anteil am Ruhestrom des Atom Lite. Durch seine Entfernung fiel der Ruhestrom um 9177 µA niedriger aus. Nach Entfernen des SGM803 Bausteins funktionierte der M5Stack Atom Lite nicht mehr, weshalb hierfür keine Daten bereitstehen.

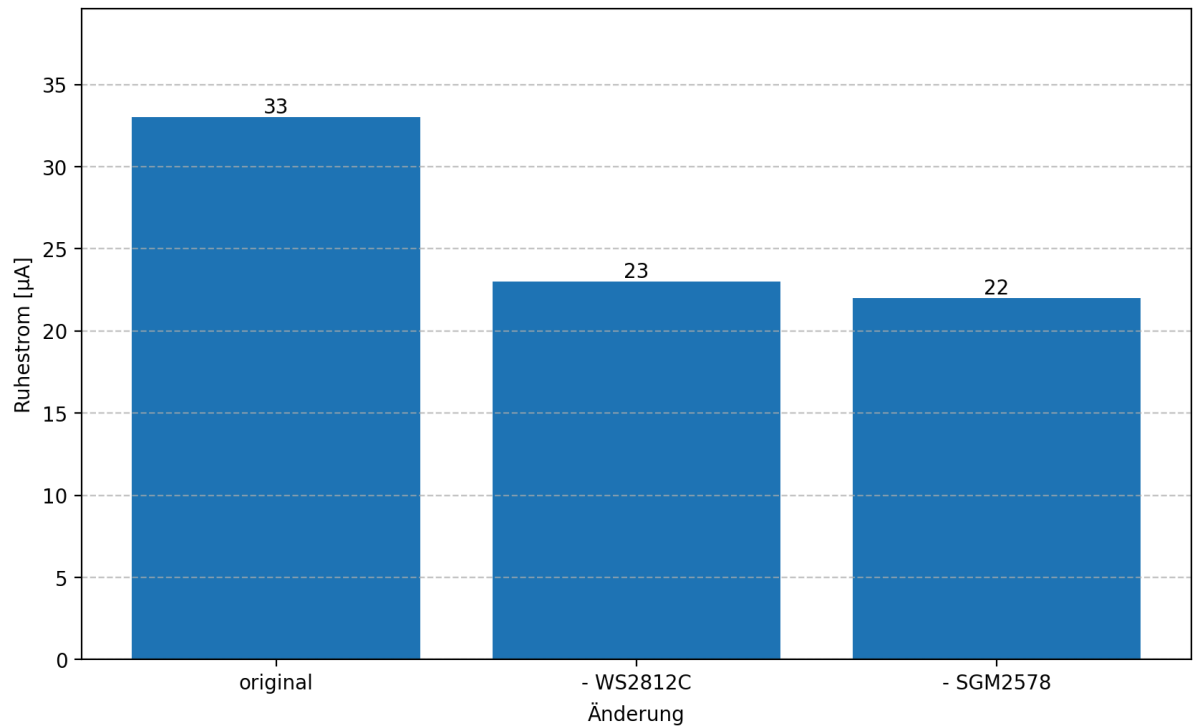


Abbildung 13: Hardwaremodifikationen des M5Stack Atom S3Lite

Der Ruhestrom des M5Stack Atom S3Lite ließ sich nochmal deutlich reduzieren, indem die LED entfernt wurde. Der Operationsverstärker hatte nur einen geringen Effekt.

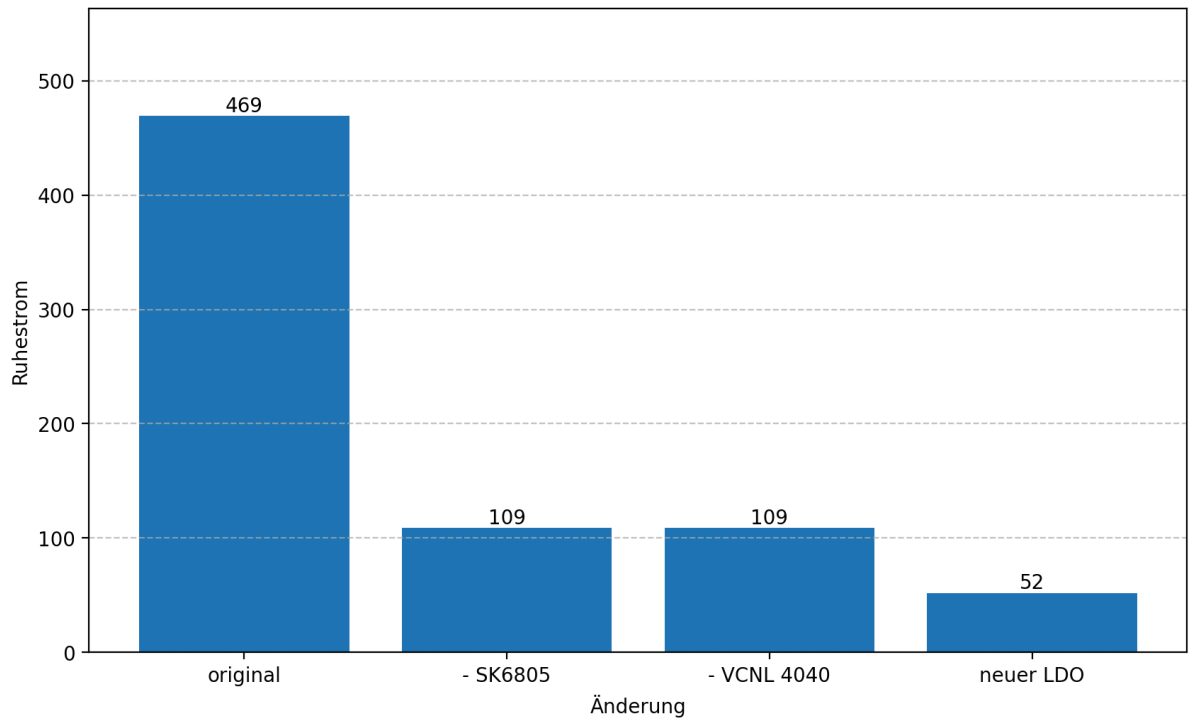


Abbildung 14: Hardwaremodifikationen des CodeCell C3 Light

Beim CodeCell C3 Light hatte das Entfernen der RGB-LED den größten Effekt. Der Helligkeits- und Entfernungssensor hatte einen nicht messbaren Energieverbrauch. Der neue LDO konnte den Ruhestrom nochmal um $57\mu\text{A}$ reduzieren.

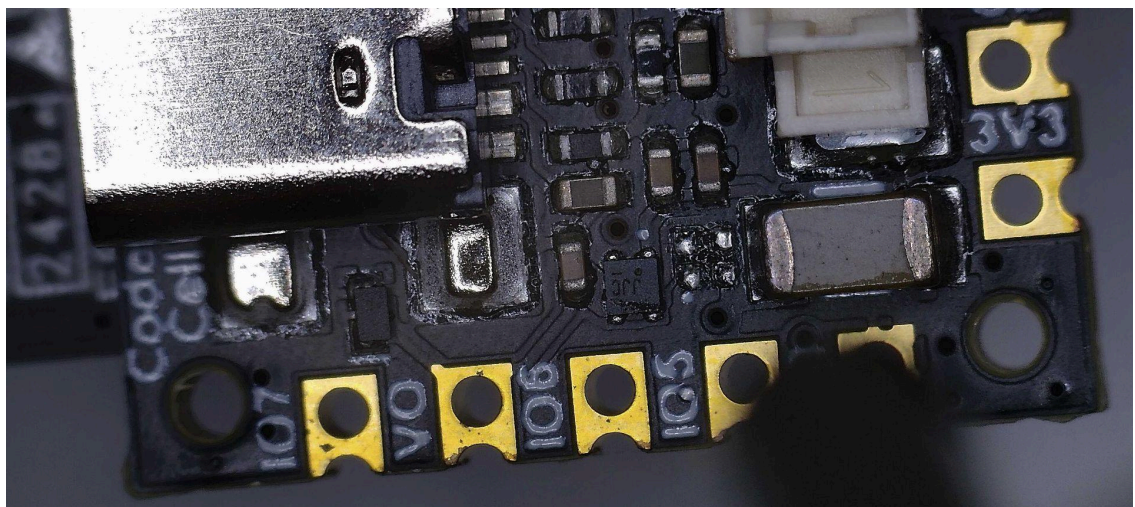


Abbildung 15: Der CodeCell C3 Light mit dem originalen LDO. Neben dem LDO wurde die LED bereits entfernt.

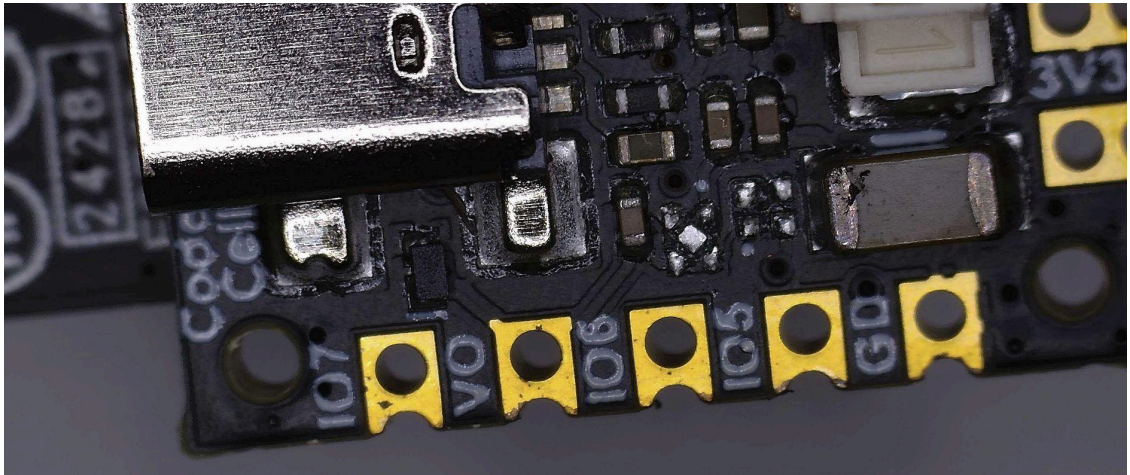


Abbildung 16: Der CodeCell C3 Light ohne LDO

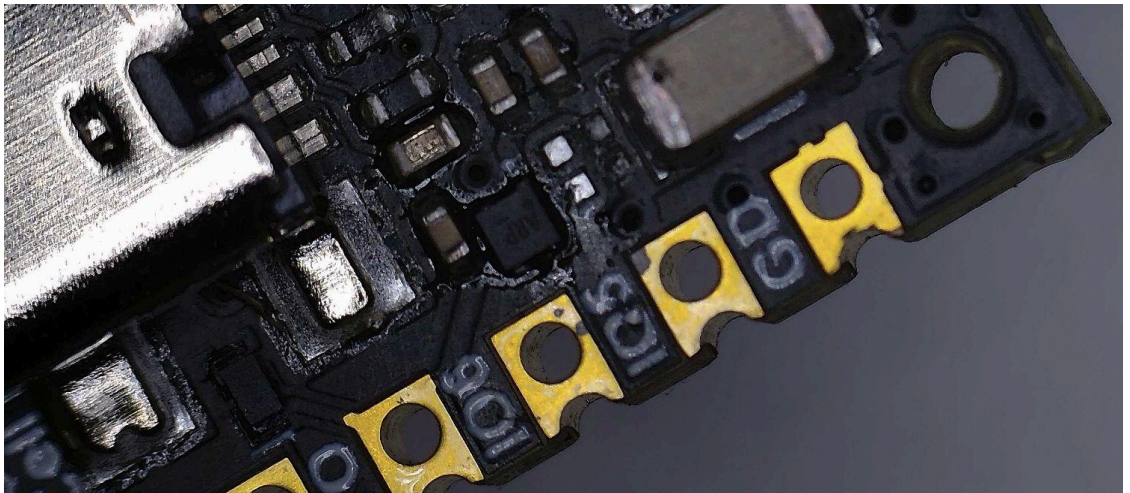


Abbildung 17: Der CodeCell C3 Light mit dem neuen LDO

4 Auswahl eines Funk-Protokolls

4.1 Das ISO-OSI Modell

Die International Organization for Standardization (ISO) hat dieses Open Systems Interconnection-Modell herausgegeben, um komplexe Kommunikationsprozesse in Netzwerken in überschaubare Schichten zu unterteilen und so zu standardisieren. Das Modell dient als theoretisches Rahmenwerk, um die Kommunikation zwischen verschiedenen Systemen zu vereinfachen und zu strukturieren. Durch die Aufteilung in sieben Schichten wird die Interoperabilität zwischen Geräten und Protokollen vereinfacht, da jede Schicht nur spezielle Aufgaben übernimmt und klar definierte Schnittstellen zu den benachbarten Schichten hat.

Die Schichten des OSI-Modells sind hierarchisch angeordnet, wobei Schicht 1 (physikalische Schicht) am nächsten an der Hardware liegt und Schicht 7 (Anwendungsschicht) am stärksten von dieser abstrahiert ist. Jede Schicht kann auf darunter liegende Schichten zurückgreifen, und kann gleichzeitig von höheren Schichten genutzt werden, um eine höhere Abstraktionsstufe zu erreichen. Das OSI-Modell wird zwar in der Praxis oft nicht 1:1 umgesetzt, aber es dient für das Konzeptionelle Verständnis von Netzwerken. Es ermöglicht eine klare Trennung von Aufgaben und erleichtert die Entwicklung und Fehlersuche in Netzwerksystemen, auch wenn nicht alle Schichten explizit abgedeckt sind.

Die Schichten des offiziellen ISO-OSI-Modells lauten wie folgt:

- **Schicht 1: Physikalische Schicht (Physical Layer)**

Verantwortlich für die Übertragung von Rohdaten (Bits) über physische Medien wie Kabel oder Funk. Hier werden elektrische Signale, Übertragungsmedien, Spannungspegel und Übertragungsraten definiert.

- **Schicht 2: Sicherungsschicht (Data Link Layer)**

Organisiert den Zugriff auf das Übertragungsmedium und sorgt für eine fehlerfreie Übertragung zwischen direkt verbundenen Geräten. Protokolle wie Ethernet oder das Point-to-Point Protocol sind hier angesiedelt²⁹.

- **Schicht 3: Vermittlungsschicht (Network Layer)**

Steuert die Weiterleitung von Datenpaketen durch das Netzwerk, z. B. durch Routing. Das Internetprotokoll (IP) ist ein zentrales Protokoll dieser Schicht, ebenso wie das Address Resolution Protocol (ARP)²⁹.

- **Schicht 4: Transportschicht (Transport Layer)**

Gewährleistet eine zuverlässige Ende-zu-Ende-Kommunikation zwischen Anwendungen³⁰. Protokolle wie TCP (Transmission Control Protocol) und UDP (User Datagram Protocol) arbeiten hier.

- **Schicht 5: Sitzungsschicht (Session Layer)**

Verwaltet die Kommunikation zwischen Anwendungen, z. B. den Aufbau, die Aufrechterhaltung und den Abbau von Verbindungen³¹. Auch die (Re)Synchronisation bei z.B einer unerwarteten Verbindungsunterbrechung wird von der Sitzungsschicht übernommen.

- **Schicht 6: Darstellungsschicht (Presentation Layer)**

Übersetzt Daten in ein Format, das von der Anwendungsschicht genutzt werden kann. Hier finden z. B. Verschlüsselung, Komprimierung oder Formatkonvertierungen statt³². Beispiele für Layer 6 Protokolle sind z.B. SSL und TLS.²⁹

- **Schicht 7: Anwendungsschicht (Application Layer)**

Bietet Schnittstellen für Anwendungen und Dienste wie HTTP, FTP oder DNS²⁹. Diese Schicht erlaubt direkte Interaktion zwischen Endnutzer und Netzwerk³³

4.2 Theoretischer Vergleich verschiedener Funkprotokolle

Neben der Entscheidung für eine Hardware-Plattform ist noch zu klären, über welches Protokoll das Klingelsignal zu übermitteln ist.

Da im Prinzip ein einzelnes Bit als Payload ausreicht, um ein Klingeln zu übertragen, ist der maximale Durchsatz eines Protokolls nicht entscheidend. Wichtiger sind dagegen eine geringe Latenz, eine hohe Zuverlässigkeit, sowie ein geringer Energieverbrauch. Letzteres wird im Wesentlichen dadurch erreicht, dass eine Übertragung schnell abgeschlossen wird und der μC wieder in den Ruhemodus wechseln kann. Bei Latenz ist nicht nur die Dauer einer einzelnen Paketübertragung gemeint, sondern auch die Zeit, die der μC braucht, um das Senden eines Pakets vorzubereiten. Es folgt eine kurze Vorstellung verschiedener gängiger Funkprotokolle.

- **UDP**

Das User Datagram Protocol ist auf Schicht 4 des ISO/OSI Modells definiert³⁴. Auf Schicht 3 nutzt es das Internet Protocol (IP). Auf Layer 2 und 1 würde in diesem Anwendungsfall WLAN (genauer: IEEE 802.11) verwendet. Da UDP ein verbindungsloses Protokoll ist, steht jedes Paket für sich selbst, es gibt keine Paketnummerierung. Außerdem gibt es keine Möglichkeit, sicherzustellen, dass ein Paket sein Ziel erreicht. Dadurch muss allerdings auf Seiten des Senders auch nicht auf eine Antwort des Empfängers gewartet werden, was die Übertragungszeit einer Transaktion verkürzt. Um die Fehlerwahrscheinlichkeit zu verringern, könnten direkt nacheinander mehrere Klingelsignale abgesetzt werden.

- **TCP**

Das Transmission Control Protocol ist ebenfalls ein Layer 4-Protokoll und stützt sich auf die gleichen Protokolle in den darunterliegenden Layern. Im Gegensatz zu UDP ist TCP allerdings ein verbindungsorientiertes Protokoll. Bevor ein Paket gesendet wird, wird zunächst durch einen Three-Way-Handshake eine Verbindung zwischen Sender und Empfänger aufgebaut. TCP stellt sicher, dass gesendete Pakete vollständig und fehlerfrei beim Empfänger ankommen. Verlorene oder nicht korrigierbar falsche Pakete werden erneut angefordert.

Für eine Klingelanlage wäre die garantierte Zustellung eines Pakets von Vorteil. Die Notwendigkeit eines Verbindungsaufbaus und das Warten auf eine Bestätigung vom Empfänger würde die Transaktion allerdings in die Länge ziehen.

- **Bluetooth**

Bluetooth³⁵ ist eine Funktechnik für Kurzstrecken. Da es Aufgaben des gesamten Osi-Iso Modell übernimmt, lässt es sich nicht ganz einfach darin eingliedern.

Auf Layer 1 nutzt es, genau wie WLAN das 2,5 GHz ISM-Band.

LoRa. Da Bluetooth ebenfalls verbindungsorientiert arbeitet, ist es nicht optimal für eine Anwendung aus dem Ruhemodus hinaus. Ein weiterer Nachteil für eine Klingelanlage ist die niedrige Reichweite von Bluetooth, die nicht vom Sender bis zu den entlegensten Empfängern reichen würde. Als Lösung für dieses Problem würde sich Bluetooth Mesh anbieten. In einer experimentellen Gegenüberstellung³⁶ wird die Reichweite von Bluetooth Mesh mit 15m angegeben. In derselben Studie wird die Dämpfung von Bluetooth-Signalen durch eine Mauer mit bis zu 15dB angegeben.

Diese Angaben lassen Zweifel daran aufkommen, ob ein Bluetooth Mesh alle Zimmer des Wohnheims erreichen würde.

- **Zigbee**

Zigbee ist ebenfalls ein drahtloser Kommunikationsstandard, der eine Mesh-Topologie unterstützt. Aber auch Stern- und Baum-Topologien sind möglich³⁷. Zigbee ist optimiert für batteriebetriebene Geräte. Die Reichweite ist mit 10-20m in Innenräumen relativ gering, was ein Problem sein könnte³⁷. Leider sind bisher nur der ESP32-C6 und der ESP32-H2 Zigbee-fähig, während die Geräte CodeCell C3 Light/Atom Lite/Atom S3Lite auf dem ESP32-C3/ESP32-Pico-D4/ESP32-S3 basiert.

Selbst wenn der Sender gegen ein Gerät ausgetauscht würde, welches Zigbee-fähig ist, so könnte der Empfänger, der auf einem ESP32-Pico-D4 basiert, dieses Signal nicht empfangen. Daher wird dieses Protokoll für die Klingelanlage nicht in Frage kommen. Das Protokoll soll aber trotzdem auf seinen Energieverbrauch untersucht werden. Dazu soll ein ESP32C6-basierter CodeCell C6 verwendet.

- **Thread**

Thread ist, genau wie Zigbee, ein drahtloses Protokoll für Mesh-Topologien, entwickelt für Low-Power-Geräte³⁸. Im Gegensatz zu Zigbee ist es aber IP-v6-basiert, wodurch es ohne Gateway auf das Internet zugreifen kann. Genau wie Zigbee sind allerdings nur der ESP32-C6, sowie der ESP32-H2 Thread-fähig. Auch für Thread soll der Stromverbrauch untersucht werden, auch wenn es nicht zum Einsatz kommen wird, da die Empfänger nicht kompatibel sind.

- **ESP-NOW**

ESP-NOW ist ein Funkprotokoll, welches eigens von der Firma Espressif für ihre Mikrocontroller ESP8266 und ESP32 entwickelt wurde. Es ist auf den Layern 3 aufwärts definiert und nutzt auf Layer 1 das Protokoll 802.11 b/g/n. Auf Layer 2 nutzt es das Point-to-Point Protocol (PPP). Nachrichten werden anhand von MAC-Adressen versendet, wobei die Übertragung verbindungslos geschieht. Es wird beworben als ein nutzerfreundliches, energiesparendes, low-latency-protokoll. In der vergleichenden Studie³⁶ wird die Line-of-Sight-Reichweite von ESP-NOW mit 185 Metern angegeben. Genau wie bei Bluetooth benötigt man keine zusätzlichen Vermittler wie Access-Points oder Router. Allerdings ist die maximale Anzahl an gepaarten Geräten bei ESP-NOW auf 20 beschränkt.

Da ESP-NOW ein punkt-zu-punkt Protokoll ist, könnte eine Klingelanlage mit ESP-NOW lediglich in einer Stern-Topologie realisiert werden. Es gibt allerdings Ansätze, basierend auf dem ESP-NOW Protokoll ein Mesh aufzubauen. Ein solches Mesh ist das zh_network³⁹ des russischen Makers Alexey Zholtikov. Bei diesem Mesh-Protokoll können beliebig viele Geräte zu einem Mesh zusammengeschlossen werden. Auch hier könnte wieder direkt nacheinander mehrere Klingelsignale abgesetzt werden, um sicherzugehen, dass ein Klingelsignal sein Ziel erreicht.

- **ESP-Wifi-Mesh**

Das ESP-Wifi-Mesh⁴⁰ ist ein Netzwerk, welches bestehende WLAN-Netzwerke um eine Mesh-Komponente erweitern kann. Es baut auf dem WiFi-Protokoll auf⁴¹. Es ist selbstformierend und selbstheilend und erlaubt auch den Zugriff auf das Internet.

- **LoRaWAN**

LoRa (Long Range) ist eine drahtlose Modulationstechnik, die von der Chirp Spread Spectrum (CSS)-Technologie abgeleitet ist. Sie kodiert Informationen auf Funkwellen mit Hilfe von Chirp-Impulsen⁴². Durch diese sehr breitbandige Übertragung kann das Signal sehr große Reichweiten erreichen, wobei die Übertragungsrate jedoch gering bleibt. Durch eine Vernetzung kann aus vielen LoRa-Modulen ein sogenanntes LoRaWAN-Netzwerk entstehen. Für LoRa braucht man spezielle Hardware. Da LoRa eine proprietäre Technik ist, und für maximale Performance auf eine externe Antenne angewiesen ist, kann ein LoRa-Modul im Vergleich zu einem reinen Mikrocontroller-Board recht teuer sein. Da jeder der Empfänger ein solches LoRa-Modul bräuchte, wird LoRa für diese Klingelanlage nicht in Betracht gezogen.

4.3 Evaluation des Energieverbrauchs Q_K der verschiedenen Funkprotokolle

Mit dem Power Profiler Kit⁹ von Nordic Semiconductors wurden die verschiedenen Funkprotokolle verglichen: Der CodeCell C3 Light wurde zunächst in den Deep-Sleep-Modus versetzt. Durch drücken eines Tasters wurde der ESP32 aufgeweckt. Dann wurde der Code ausgeführt, der so schnell wie möglich eine Nachricht absetzte, bevor sich der ESP32 wieder in den Deep-Sleep-Modus versetzte. Als Maß für die verbrauchte Energie wurde das Integral des Stromverbrauchs, vom Aufwecken aus dem Deep-Sleep-Modus, bis zum erneuten Beginn des Deep-Sleep-Modus, gemessen. Da die Versorgungsspannungen konstant bei 5V lagen, ist die verbrauchte Ladung ein gutes Maß für die verbrauchte Energie.

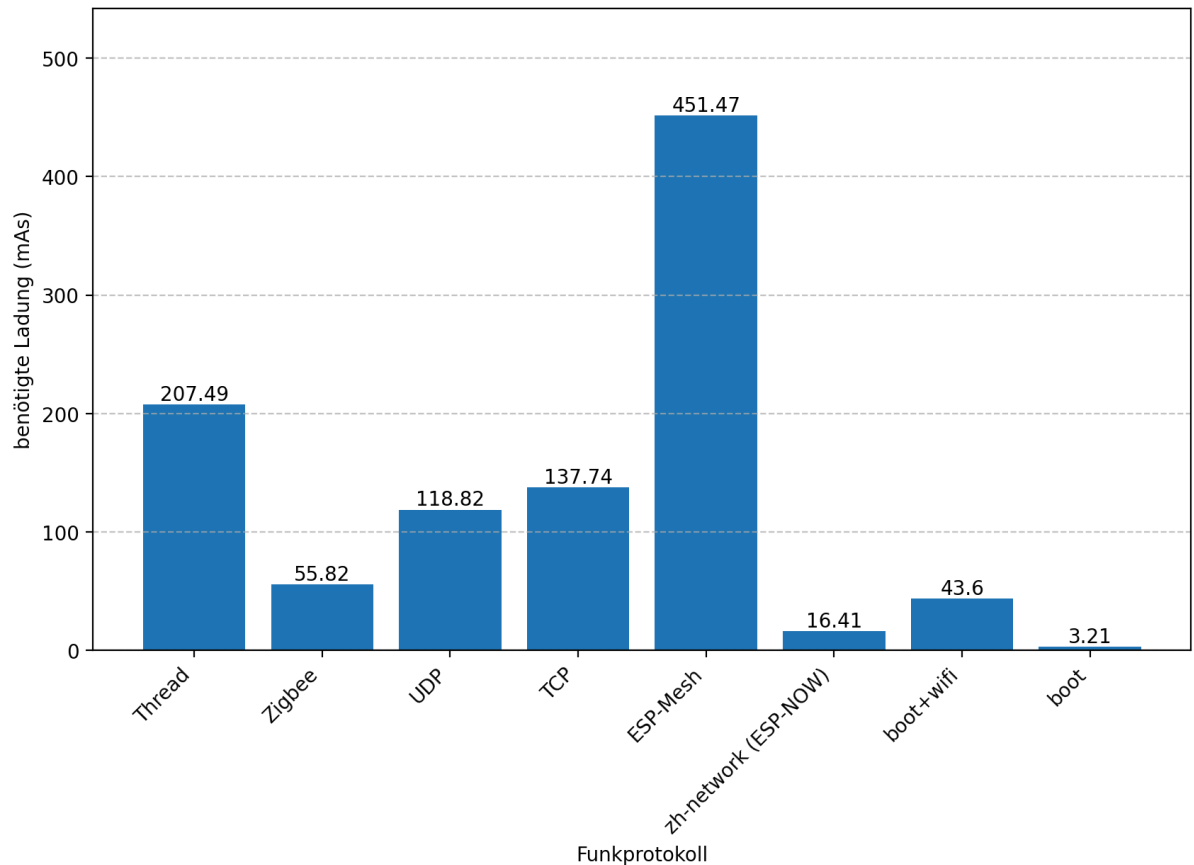


Abbildung 18: Die Energieverbräuche verschiedener Funkprotokolle im Vergleich

Als Baseline diente ein Programm, welches lediglich aus dem Deep-Sleep-Modus aufwachte und sich sofort wieder in den Deep-Sleep-Modus begab. Ein Programm, welches sich nach dem Aufwachen sofort mit einem WiFi-Netz verbindet, wurde ebenfalls zum Vergleich herangezogen.

Der Energieverbrauch von Thread war im Vergleich zu Zigbee überraschend hoch, was darauf zurückzuführen war, dass der ESP32 sehr lange brauchte, um sich nach dem Booten mit dem Thread Netzwerk zu verbinden. Bei Zigbee gelang das deutlich schneller, weshalb der Energieverbrauch hier deutlich geringer war. UDP und TCP performen ähnlich, was auch nicht überrascht, da sie auf den OSI-Ebenen 1-3 die gleichen Protokolle nutzen. Der Verbindungsaufbau mittels ESP-Mesh dauerte sehr lange, weshalb der Energieverbrauch hier sehr hoch war. Das ESP-NOW-basierte zh-network performte am besten, mit nur rund 16 mAh verbrauchter Ladung.

5 Interpretation der ermittelten Werte

Anhand von [Abbildung 3](#) liegt es nahe, sich für einen M5Stack Atom S3Lite mit Ruhestrom $I_s = 29\mu A$ zu entscheiden. Als Funkstandard bietet sich ESP-NOW mit $Q_K = 17000\mu As$ benötigter Ladung pro Klingelvorgang ([Abbildung 4](#)). Nehmen wir an, dass die Klingel 5 mal am Tag betätigt wird, so entfallen auf das aktive Klingeln jeden Tag $5 \cdot Q_K = 85000\mu As$. Der passive Stromverbrauch im Deep-Sleep-Modus verbraucht dagegen jeden Tag $I_s \cdot 3600 \cdot 24 = 2505600\mu As$. Der passive Energieverbrauch ist also rund dreißigmal größer als der Stromverbrauch für das Klingeln.

Es wäre also durchaus wünschenswert, den Standbystrom I_s weiter zu reduzieren. Da sich hier auf kommerziell erhältliche Hardware beschränkt wurde, müssen wir uns mit einem I_s von $29\mu A$ zufriedengeben.

5.1 Erreichbare Lebensdauern

Das für den M5Stack AtomS3Lite vom Hersteller vorgesehene Batteriemodul "Atomic Battery Base"⁴³ verfügt über einen LiPo-Akku von 200mAh. Für eine vereinfachte Analyse gehen wir davon aus, dass der im Batteriemodul vorhandene Boost-Converter und der im AtomS3Lite verbaute Buck-Converter den vom ESP32 verbrauchten Strom Eins zu Eins an diesen LiPo-Akku weitergeben.

Bei weiterhin angenommenen 5 Klingelvorgängen pro Tag kommen wir mit AtomS3Lite und ESP-NOW auf einen täglichen Strombedarf von $Q_T = 5 \cdot Q_K + I_s \cdot 3600 \cdot 24 = 2586\text{ mAs}$

Mit einer $200\text{mAh} = 720000\text{ mAs}$ großen Batterie würde die Klingel also rund $720000/2586 = 278$ Tage durchhalten.

Diese grobe Überschlagsrechnung berücksichtigt keinerlei Selbstentladung der Batterie. Außerdem sind die Verluste der Boost-Wandler der Batterie nicht mit eingerechnet. Die tatsächliche Lebensdauer der Batterie dürfte also vermutlich geringer ausfallen, man könnte sie auf sechs Monate schätzen.

5.2 Ausblick: Maßnahmen zur Verlängerung der Batterielebensdauer

Da es doch relativ unpraktikabel wäre, die Batterie alle sechs Monate aufzuladen, sollte die Batterielebensdauer perspektivisch erhöht werden. Dazu gibt es verschiedene Möglichkeiten:

- **Größerer Akku:**

Für eine längere Laufzeit können wir den LiPo-Akku der Atomic Battery Base beispielsweise durch einen 1800 mAh LiPo-Akku austauschen. So verlängern wir die überschlagene Akkulaufzeit auf rund 2500 Tage oder 7 Jahre. Hierfür müsste aber ein neues Gehäuse für die größere LiPo-Zelle her.

- **Energy Harvesting:**

Mit preisgünstig erhältlichen Solarmatten⁴⁴ (~10€) lassen sich beispielsweise Smartphones mit 5 bis 10 Watt laden. Da die Atomic Battery Base⁴³ sowieso nur mit maximal 223 mA (1.115 W) lädt, würde also eine kurze Hellperiode ausreichen, um den 200mAh-Akku wieder aufzuladen. Da der Bereich vor der Haustür allerdings kein direktes Sonnenlicht erhält, müsste die Solarplatte die Batterie auch mit indirektem Sonnenlicht laden können. Leider kann die Atomic Battery Base nur geladen werden, wenn sie ausgeschaltet ist. Die M5Stack TailBat kann dagegen auch im laufenden Betrieb geladen werden. Mit einem solchen Setup könnte die Klingelanlage auch ohne Batteriewechsel dauerhaft betrieben werden, vorausgesetzt, die Solarplatte bekommt genug Licht, um einen Ladestrom zu erzeugen

- **Power Latching Circuit:**

Ein Power-Latching-Circuit, wie ihn der Picoclick und der Objex Link einsetzen, könnte auch nachträglich nachgerüstet werden. Notwendig ist dafür nur ein Bauteil, welches die Stromzufuhr des ESP32 schalten kann. Dieses Bauteil muss sowohl von einem externen Taster, als auch vom ESP32 selbst aktiviert werden können. Im Picoclick wird diese Funktion vom LDO übernommen, der einen Enable-Eingang besitzt. Es kommt allerdings auch ein Pass-Transistor in Frage, wie in Abb gezeigt. Ein Drücken auf den Taster aktiviert über den npn-Transistor den P-Channel-MOSFET, sodass der Mikrocontroller einschalten kann. Über einen GPIO kann dieser den npn-Transistor selbst leitend halten, auch wenn der Taster losgelassen wird. Ist das Klingelsignal versendet worden, kann sich der Mikrocontroller über den GPIO selbst abschalten.

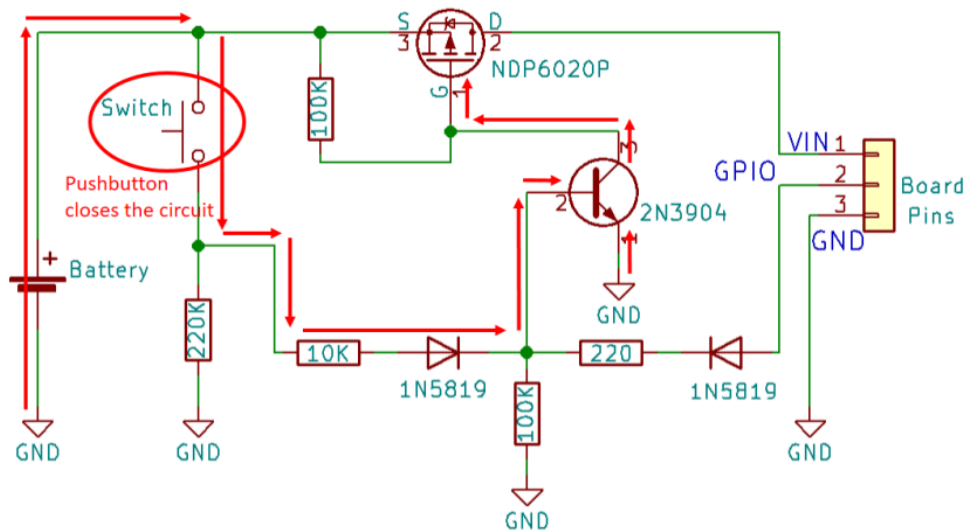


Abbildung 19: Ein Power-Latching-Circuit im wesentlichen bestehend aus zwei Transistoren⁴⁵

Da sich in diesem Projekt allerdings auf kommerziell erwerbliche Hardware beschränkt werden sollte, wird dieser Weg nicht weiter verfolgt.

- **An-Aus-Taster:**

Um den Power-Latching-Circuit zu vereinfachen, könnte man auch die Transistoren weglassen und die Versorgungsspannung des Mikrocontrollers direkt schalten.

Da der ESP32 mit ESP-Now lediglich 300ms braucht, um zu booten und die Nachricht abzusetzen, müsste der Taster auch nur 300 ms lang gehalten werden.

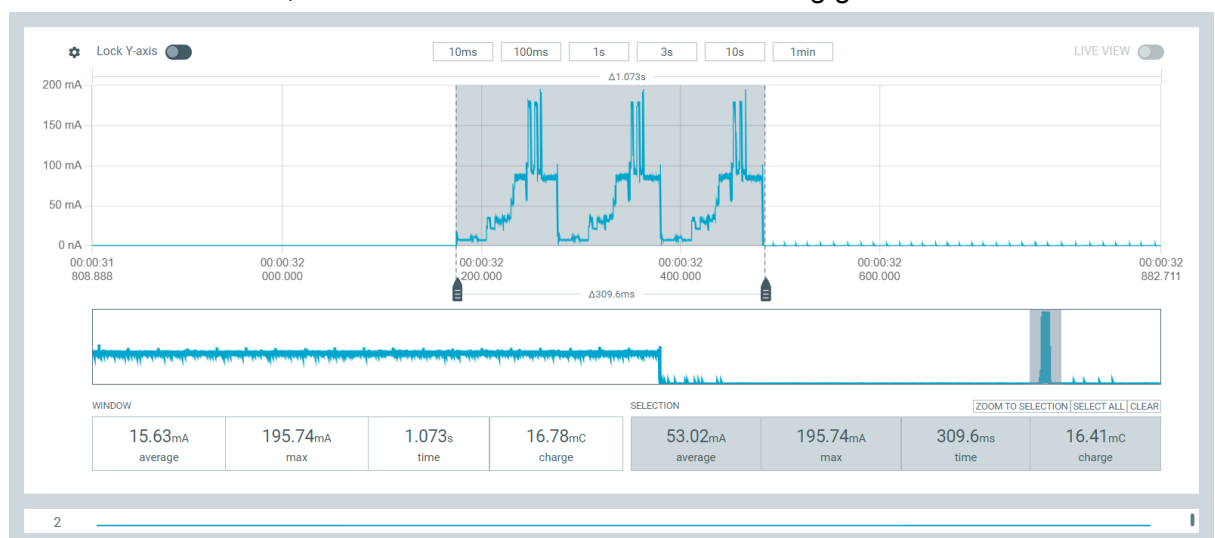


Abbildung 20: Das senden des Klingelsignals mit ESP-now dauert nur etwas mehr als 300

6 Zusammenfassung

Im Rahmen dieser Studienarbeit wurde eine energiesparende, funkbasierte Klingelanlage für das ESA-Wohnheim der RPTU Kaiserslautern konzipiert und prototypisch umgesetzt. Ziel war es, eine nachrüstbare Lösung zu entwickeln, die ohne Kabelverlegung auskommt.

6.1 Ergebnisse und Erkenntnisse

Als Hardware-Plattform erwies sich der M5Stack Atom S3Lite aufgrund seines geringen Ruhestroms von nur 29 μA im Deep-Sleep-Modus als am besten geeignet. Durch gezielte Modifikationen – wie das Entfernen unnötiger Komponenten (z. B. LEDs) – konnte der Energieverbrauch weiter optimiert werden.

Das ESP-NOW-basierte zh-network zeigte sich als das effizienteste Protokoll mit einem Energieverbrauch von nur 17 mAs pro Klingelvorgang. Im Vergleich zu Alternativen wie TCP, UDP oder Thread ermöglichte es eine schnelle Übertragung bei minimalem Stromverbrauch. Es ergibt sich eine theoretische Batterielebensdauer von ca. 278 Tagen mit dem standardmäßigen 200-mAh-Akku. Durch den Einsatz eines größeren Akkus (z. B. 1800 mAh) oder Energy Harvesting (z. B. Solarmodule) ließe sich die Laufzeit weiter verlängern.

6.2 Herausforderungen

Die Implementierung der Funkprotokolle stützte sich auf Beispielcodes, die teilweise für andere Boards gedacht waren. Sie basierten auf Bibliotheken, die möglicherweise nicht zu 100% auf maximales Energiesparen ausgelegt waren. Deshalb ist es gut möglich, dass z.B. eine energiesparendere Implementierung des Thread-Protokolls möglich gewesen wäre. Die Reduzierung des Ruhestroms auf kommerziell erhältlichen Boards stieß an Grenzen.

Spezialisierte Low-Power-Boards (z. B. mit Power-Latch-Schaltungen) wären ideal, waren jedoch zum Zeitpunkt der Arbeit nicht verfügbar. Auch die Energieversorgung mit den Boost- und Buck Convertern der kommerziell verfügbaren Hardware ist nicht optimal.

6.3 Ausblick und Empfehlungen

Die entwickelte Lösung bietet eine praktikable und kostengünstige Basis für das Retrofitting des Wohnheims. Für eine Implementierung empfehlen sich folgende Schritte:

In einer Pilotphase sollte die Klingelanlage unter Realbedingungen getestet werden, um die tatsächliche Batterielebensdauer und die Zuverlässigkeit der Funkübertragung zu validieren.

Diese Pilotphase könnte mit einigen wenigen Empfängern durchgeführt werden, um die Kosten gering zu halten. Bei Erfolg könnten weitere Empfänger hinzugefügt werden.

Das System könnte um eine Solarzelle ergänzt werden, um Wartungsintervalle zu minimieren. Bei Bedarf könnte die Software der Empfänger erweitert werden, um dem Sender oder anderen Empfängern zu signalisieren, dass jemand die Tür öffnen wird.

Optional könnte der Sender erweitert werden, um gezielte Klingelsignale an einzelne Zimmer zu ermöglichen.

7 Fazit

Die Studienarbeit zeigt, dass sich mit kommerziell verfügbarer Hardware und Open-Source-Protokollen eine funktionsfähige, energiesparende Klingelanlage realisieren lässt. Durch die Kombination von ESP32-basierten Boards, offenen Funkprotokollen und ein paar gezielten Hardware-Modifikationen wurde ein Kompromiss zwischen Energieeffizienz, Reichweite und Praktikabilität gefunden. Die Ergebnisse unterstreichen das Potenzial von Low-Power-IoT-Lösungen für Amateurprojekte in Bestandsgebäuden – auch dort, wo die Infrastruktur fehlt.

8 Quellenverzeichnis

- (1) *ESP32 Power Consumption and Sleep Modes*. Arrow.com.
<https://www.arrow.com/en/research-and-events/articles/esp32-power-consumption-can-be-reduced-with-sleep-modes> (accessed 2026-02-24).
- (2) *nRF52840 - Nordic Semiconductor*. <https://www.nordicsemi.com/Products/nRF52840> (accessed 2026-02-27).
- (3) *Deep Sleep? RP2040 for 24/7 battery-powered device - Page 1*.
<https://www.eevblog.com/forum/microcontrollers/deep-sleep-rp2040-for-247-battery-powered-device/> (accessed 2026-02-27).
- (4) Lee, J. *ESP-Knob: A multifunctional smart controller with the ESP32-H2*. Developer Portal. <https://developer.espressif.com/blog/2025/03/esp32h2-knob/> (accessed 2026-02-11).
- (5) *ESP32 Ethernet Unit with PoE*. m5stack-store.
<https://shop.m5stack.com/products/esp32-ethernet-unit-with-poe> (accessed 2026-03-27).
- (6) *Atomic Battery Base*. <https://docs.m5stack.com/en/atom/Atomic%20Battery%20Base> (accessed 2026-03-27).
- (7) *ATOM TailBat - Battery Accessory for ATOM*. m5stack-store.
<https://shop.m5stack.com/products/atom-tailbat> (accessed 2026-03-27).
- (8) LABS, O. *OBJEX LABS | Behind Every Revolution*. OBJEX LABS. <https://objexlabs.com/> (accessed 2026-02-24).
- (9) *Get started with the Power Profiler Kit II (PPK2)*.
<https://www.nordicsemi.com/Products/Development-hardware/Power-Profiler-Kit-2/GetStarted> (accessed 2026-03-27).
- (10) *TPS7A20 data sheet, product information and support | TI.com*.
https://www.ti.com/product/TPS7A20?utm_source=google&utm_medium=cpc&utm_campaign=app-null-null-GPN_EN-cpc-pf-google-eu_en_cons&utm_content=TPS7A20&ds_k=TPS7A20&DCM=yes&gclid=aw.ds&gad_source=1&gad_campaignid=8753988867&gc

- lid=CjwKCAiA2PrMBhA4EiwAwphYc8lfXN1a3iGsk1kucQIPXrng3N9gu0Wnb06tEr7siYE
ACCKBE_XSWWhoCjfeQAvD_BwE (accessed 2026-02-25).
- (11) *UCLAMP3311Z. TFT Semtech | Mouser.* Mouser Electronics.
<https://www.mouser.de/ProductDetail/Semtech/UCLAMP3311Z.TFT?qs=rBWM4%252BvDhlcs%2FUePeLOH2w%3D%3D> (accessed 2026-02-25).
- (12) *GS321-TR | Gainsil | Price | In Stock.* LCSC Electronics.
<https://www.lcsc.com/product-detail/C157695.html> (accessed 2026-02-26).
- (13) *WS2812C-2020 Worldsemi - LEDs - Distributoren, Preisvergleich und Datenblätter | Octopart-Komponentensuche.*
https://octopart.com/de/part/worldsemi/WS2812C-2020?gad_source=1&gad_campaignid=20639798297&gclid=Cj0KCQiAtfXMBhDzARIsAJ0jp3CwigOnZmfs74POX7k48Px2xXG7lgTGCoGefg_4MlhVZ3pmQfzXqGwaAjPWEALw_wcB (accessed 2026-02-24).
- (14) *OPSCO 10 x RGBW LED with IC SK6812 SMD 3535 6 PIN 2700-3200K SK6812MI, 5,45 €.* OPSCO 10 x RGBW LED with IC SK6812 SMD 3535 6 PIN 2700-3200K SK6812MI, 5,45 €.
<https://eckstein-shop.de/OPSCO10xRGBWLEDwithICSK6812SMD35356PIN2700-3200KSK6812MINI-RGBW-WS-10> (accessed 2026-02-24).
- (15) *AtomS3-Lite.* <https://docs.m5stack.com/en/core/AtomS3%20Lite> (accessed 2026-02-24).
- (16) *SK6805-EC10-000 | OPSCO Optoelectronics | Price | In Stock.* LCSC Electronics.
<https://www.lcsc.com/product-detail/C22394946.html> (accessed 2026-02-24).
- (17) *Clark, L. Overview | Adafruit CH552 QT Py | Adafruit Learning System.*
<https://learn.adafruit.com/adafruit-ch552-qt-py/overview> (accessed 2026-02-24).
- (18) *Atom-Lite.* <https://docs.m5stack.com/en/core/ATOM%20Lite> (accessed 2026-02-24).
- (19) *AtomS3-Lite.* <https://docs.m5stack.com/en/core/AtomS3%20Lite> (accessed 2026-02-24).
- (20) *GS321-TR | Gainsil | Price | In Stock.* LCSC Electronics.
<https://www.lcsc.com/product-detail/C157695.html> (accessed 2026-02-24).
- (21) *SY8089A | TECH PUBLIC | Price | In Stock.* LCSC Electronics.
<https://www.lcsc.com/product-detail/C41348273.html> (accessed 2026-02-24).
- (22) *CodeCell/hardware/CodeCell C3 Light at main · microbotsio/CodeCell.* GitHub.
<https://github.com/microbotsio/CodeCell/tree/main/hardware/CodeCell%20C3%20Light> (accessed 2026-02-24).
- (23) *VCNL4040 Proximity Ambient Light Digital Sensor.* DigiKey.
<https://www.digikey.de/de/ptm/v/vishay-semiconductor-opto-division/vcnl4040-proximity-ambient-light-digital-sensor> (accessed 2026-02-24).
- (24) *ESD5Z.* onsemi.
<https://www.onsemi.com/products/discrete-power-modules/esd-protection-diodes/esd5z> (accessed 2026-02-24).
- (25) *SP3031-01ETG Littelfuse | Mouser.* Mouser Electronics.
<https://www.mouser.de/ProductDetail/Littelfuse/SP3031-01ETG?qs=B%2FZyIADI9o%252B%2FfpJwZJPmLg%3D%3D> (accessed 2026-02-24).
- (26) *SGM2578 data sheet and product information - SGMICRO.*
<https://www.sg-micro.de/product/SGM2578> (accessed 2026-02-24).
- (27) *SGM803-SXN3L/TR | SGMICRO | Price | In Stock.* LCSC Electronics.
<https://www.lcsc.com/product-detail/C99577.html> (accessed 2026-02-24).
- (28) *NCP177AMX180TCG.* DigiKey Electronics.

- <https://www.digikey.de/de/products/detail/onsemi/NCP177AMX180TCG/5848192>
(accessed 2026-02-25).
- (29) *Beckhoff Information System - English.*
https://infosys.beckhoff.com/english.php?content=../content/1033/tf6310_tc3_tcpip/84246923.html&id= (accessed 2026-02-24).
- (30) *Transport Layer in OSI Model.* GeeksforGeeks.
<https://www.geeksforgeeks.org/computer-networks/transport-layer-in-osi-model/>
(accessed 2026-02-24).
- (31) *Sitzungsschicht: Alles Wichtige über die fünfte Ebene des OSI-Modells.* IONOS Digital Guide. <https://www.ionos.de/digitalguide/server/knowhow/sitzungsschicht/>
(accessed 2026-02-24).
- (32) *Presentation Layer.* ComputerWeekly.de.
<https://www.computerweekly.com/de/definition/Presentation-Layer> (accessed 2026-02-24).
- (33) *Application Layer in OSI Model.* GeeksforGeeks.
<https://www.geeksforgeeks.org/computer-networks/application-layer-in-osi-model/>
(accessed 2026-02-24).
- (34) *UDP - User Datagram Protocol.*
<https://www.elektronik-kompodium.de/sites/net/0812281.htm> (accessed 2026-02-14).
- (35) *Bluetooth 2.0 / 2.1.* <https://www.elektronik-kompodium.de/sites/kom/1010121.htm>
(accessed 2026-02-14).
- (36) Eridani, D.; Rochim, A. F.; Cesara, F. N. Comparative Performance Study of ESP-NOW, Wi-Fi, Bluetooth Protocols Based on Range, Transmission Speed, Latency, Energy Usage and Barrier Resistance. In *2021 International Seminar on Application for Technology of Information and Communication (iSemantic)*; 2021; pp 322–328.
<https://doi.org/10.1109/iSemantic52711.2021.9573246>.
- (37) *What Is Zigbee And How Does It Work? - Aqara.*
<https://www.aqara.com/eu/blog-eu/what-is-zigbee-and-how-does-it-work-2/> (accessed 2026-02-26).
- (38) *Was ist Thread?.* OpenThread. <https://openthread.io/guides/thread-primer?hl=de>
(accessed 2026-02-26).
- (39) Zholtikov, A. aZholtikov/Zh_network, 2026. https://github.com/aZholtikov/zh_network
(accessed 2026-02-15).
- (40) *ESP-WIFI-MESH | Espressif Systems.*
<https://www.espressif.com/en/products/sdks/esp-wifi-mesh/overview> (accessed 2026-02-15).
- (41) *ESP-WIFI-MESH - ESP32 - — ESP-IDF Programming Guide v4.4.1 documentation.*
<https://docs.espressif.com/projects/esp-idf/en/v4.4.1/esp32/api-guides/esp-wifi-mesh.html> (accessed 2026-02-15).
- (42) *What are LoRa and LoRaWAN?.* The Things Network.
<https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/> (accessed 2026-02-27).
- (43) *Atomic Battery Base.* <https://docs.m5stack.com/en/atom/Atomic%20Battery%20Base>
(accessed 2026-02-27).
- (44) *3W 5V Portable Solar Panel Charger Photovoltaic Cells for Solar Light for 3.7V Battery 3-5V Battery/Phone Charger - AliExpress 44.*
https://de.aliexpress.com/item/1005007402518746.html?spm=a2g0o.productlist.main.53.54e07f2E7f2EwS&algo_pvid=6c2f869a-212c-4a9b-902d-0dc583042aa2&algo_exp_id=

6c2f869a-212c-4a9b-902d-0dc583042aa2-52&pdp_ext_f=%7B%22order%22%3A%22630%22%2C%22eval%22%3A%221%22%2C%22fromPage%22%3A%22search%22%7D&pdp_npi=6%40dis%21EUR%212.49%212.49%21%21%2119.30%2119.30%21%40210384b917738648225092321e1af4%2112000044409450917%21sea%21DE%21803566195%21X%211%210%21n_tag%3A-29919%3Bd%3Aa0279dbc%3Bm03_new_user%3A-29895&curPageLogUid=0XVwB2tWecXy&utparam-url=scene%3Asearch%7Cquery_from%3A%7Cx_object_id%3A1005007402518746%7C_p_origin_prod%3A (accessed 2026-03-28).

(45) Santos, R. *Latching Power Switch Circuit (Auto Power Off Circuit) | Random Nerd Tutorials.*

<https://randomnerdtutorials.com/latching-power-switch-circuit-auto-power-off-circuit-esp32-esp8266-arduino/> (accessed 2026-04-02).

Hilfsmittelverzeichnis

Bei der Erstellung der Arbeit habe ich Systeme benutzt, die auf künstlicher Intelligenz (KI) basieren. Diese sind in unten stehender Liste aufgeführt sind:

Ich erkläre, dass ich

- ▣ mich aktiv über die Leistungsfähigkeit und Beschränkungen der genannten KI-Systeme informiert habe,
- ▣ die aus den angegebenen KI-Systemen übernommenen Passagen gekennzeichnet habe,
- ▣ überprüft habe, dass die mithilfe der genannten KI-Systeme generierten und von mir übernommenen Inhalte faktisch richtig sind,
- ▣ mir bewusst bin, dass ich als Autor:in dieser Arbeit die Verantwortung für die in ihr gemachten Angaben und Aussagen trage.

Arbeitsschritt	Eingesetzte(s) KI-System(e)	Beschreibung der Verwendungsweise
Generierung von Ideen und Konzeption der Arbeit		
Literatursuche		
Literaturanalyse		
Literaturverwaltung und Zitationsmanagement		
Auswahl von Methoden und Modellen		
Datensammlung und -analyse		
Generierung von Programmcodes	Le Chat	Bei der Fehlerbehandlung: Entschlüsselung der Fehlermeldung und Vorschläge zur Korrektur
Erstellung von Visualisierungen		
Interpretation und Validierung		
Strukturierung des Texts der Arbeit		
Formulierung des Texts der Arbeit		
Übersetzung des Texts der Arbeit		
Redigieren des Texts		

Technische Hochschule mannheim	Studienarbeit Meyer, Marcel 3008377 Studiengang: ETBU
---	--

Vorbereitung der Präsentation des Texts		
Sonstiges		

Heidelberg, 25.08.2025



Ort, Datum, Unterschrift

Anhang

Alle Quellcodes und sind unter <https://github.com/meyerml/studienarbeit> verfügbar.